
Algoritmos y Estructura de Datos II

Lic. en Sistemas de Información
Ing. en Sistemas de Información
Ing. Mecatrónica

2024

Algoritmo

Definición

Secuencia finita de instrucciones, reglas o pasos que describen en forma precisa las operaciones que una computadora debe realizar para llevar a cabo una tarea en tiempo finito [Knuth, 1968].

Descripción de un esquema de comportamiento expresado mediante un repertorio finito de acciones y de informaciones elementales, identificadas, bien comprendidas y realizables a priori. Este repertorio se denomina léxico [Scholl, 1988].

Características de un Algoritmo

- **Ser preciso:** esto significa que las operaciones o pasos del algoritmo deben desarrollarse en un orden estricto, ya que el desarrollo de cada paso debe obedecer a un orden lógico.
 - **Ser definido.** Ya que en el área de programación, el algoritmo es el paso previo fundamental para desarrollar un programa, es necesario tener en cuenta que el computador solo desarrollará las tareas programadas y con los datos suministrados.
-

Características de un Algoritmo

- **Ser finito:** esta característica implica que el número de pasos de un algoritmo, por grande y complicado que sea el problema que soluciona, debe ser limitado.
 - **Presentación formal:** para que el algoritmo sea entendido por cualquier persona interesada es necesario que se exprese en alguna de las formas comúnmente aceptadas.
 - **Corrección:** el algoritmo debe ser correcto, es decir debe satisfacer la necesidad o solucionar el problema para el cual fue diseñado.
-

Características de un Algoritmo

- **Eficiencia:** hablar de eficiencia o complejidad de un algoritmo es evaluar los recursos de cómputo que requiere para almacenar datos y para ejecutar operaciones frente al beneficio que ofrece. En cuanto menos recursos requiere será más eficiente el algoritmo.
-

Variables o Identificadores

Una variable es, básicamente, un nombre que hace referencia a un valor. Dicho nombre no puede empezar por números, contener algunos símbolos prohibidos ni ser igual a algunas palabras reservadas.

a = 10

Sentencia

- Sección de código que representa un comando o acción.
 - Es una instrucción que el lenguaje de programación que estemos usando puede ejecutar.
-

Lenguaje de Programación C

- El lenguaje de programación C fue creado por Brian Kernighan y Dennis Ritchie a mediados de los años 70.
 - Todo programa de C consta, básicamente, de un conjunto de funciones, y una función llamada ***main***, la cual es la primera que se ejecuta al comenzar el programa, llamándose desde ella al resto de funciones que componen nuestro programa.
 - El lenguaje C posee un número reducido de palabras reservadas (tan solo 32) que define el standard ANSI-C.
-

Estructura de un programa en C

```
#include<stdio.h> //directivas de inclusion  
  
int main(){ //funcion principal  
    printf("Hola Mundo!!");  
    return 0;  
}
```

Declaración de Variables

En C, las variables pueden ser declaradas en cuatro lugares del programa:

- Fuera de todas las funciones del programa, son las llamadas variables globales, accesibles desde cualquier parte del programa.
 - Dentro de una función, son las llamadas variables locales, accesibles tan solo por la función en las que se declaran.
-

Declaración de Variables

- Como parámetros a la función, accesibles de igual forma que si se declaran dentro de la función.
 - Dentro de un bloque de código del programa, accesible tan solo dentro del bloque donde se declara. Esta forma de declaración puede interpretarse como una variable local del bloque donde se declara.
-

Tipos y Tamaños

C tiene pocos tipos de datos

- **char** Un byte. Contiene un carácter (o un número entre 0 y 255)
 - **int** Un entero.
 - **float** Un real
 - **double** Un real en doble precisión
-

Declarar una variable en C

```
#include<stdio.h>
```

```
int main(){
```

```
    // declarar una variable de tipo entera
```

```
    int var1;
```

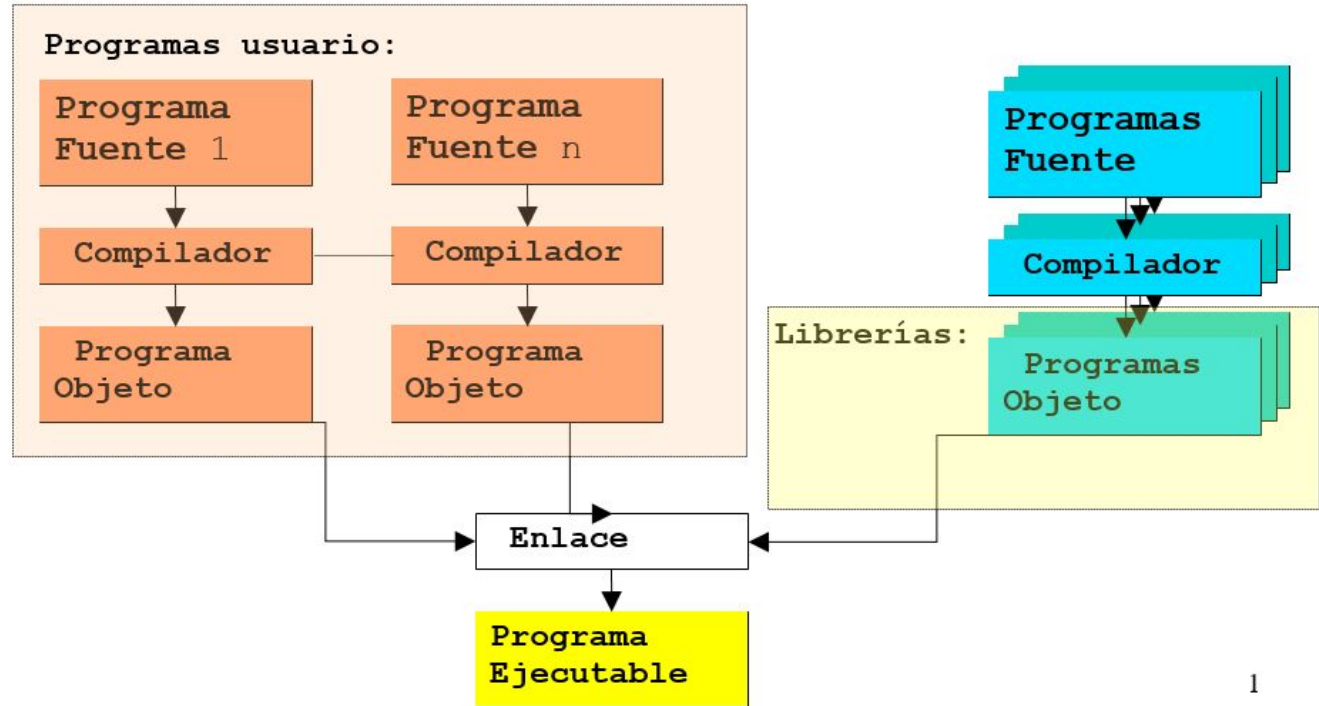
```
    // Seteo un valor a una variable
```

```
    var1 = 100;
```

```
    return 0;
```

```
}
```

Proceso de Compilación de un Programa en C



Operadores

Los operadores son necesarios para realizar operaciones. Distinguimos entre operadores **aritméticos**, **relacionales** y **lógicos**. Analizaremos también operadores aritméticos, operadores de incremento y decremento, y el operador coma.

Operadores

<i>Operador aritmético</i>	<i>Operación</i>
+	Suma
-	Resta
*	Multiplicación
/	División
%	Módulo(residuo)

Operadores - jerarquía de los operadores aritméticos

<i>Operador</i>	<i>Operación</i>
<i>*</i> , <i>/</i> , <i>%</i>	Multipliación, división, módulo
<i>+</i> , <i>-</i>	Suma, resta

Operadores de incremento y decremento

<i>Operador</i>	<i>Operación</i>	<i>Ejemplos</i>	<i>Resultados</i>
++	Incremento	x = 7;	x = 7
		y = x++;	y = 7
			x = 8
		x = 7;	x = 7
		y = ++x;	y = 8
			x = 8
--	Decremento	x = 6;	x = 6
		y = x--;	y = 6
			x = 5
		x = 6;	x = 6
		y = --x;	y = 5
			x = 5

Operadores Relacionales

<i>Operador relacional</i>	<i>Operación</i>	<i>Ejemplos</i>	<i>Resultados</i>
=	Igual a	res = 'h' == 'p';	res = 0
!=	Diferente de	res = 'a' != 'b';	res = 1
<	Menor que	res = 7 < 15;	res = 1
>	Mayor que	res = 22 > 11;	res = 1
<=	Menor o igual que	res = 15 <= 2;	res = 0
>=	Mayor o igual que	res = 35 >= 20;	res = 1

Operadores Lógicos

Operador Lógico	Operación
!	Negación
&&	AND
	OR

Estructura Condicional - IF

La estructura selectiva **if** permite que el flujo del programa siga por un camino específico *si se cumple una condición* determinada. si al evaluar la condición el resultado es verdadero, entonces se sigue por un camino específico. En cambio *si la evaluación es falsa* se puede seguir por otro camino, si lo he definido de esa manera.

Estructura Condicional - IF

/* El conjunto de instrucciones muestra la sintaxis de la estructura if en el lenguaje C. */

...

```
if (<condición>)
```

```
    <sentencia>;
```

Estructura Condicional - IF

```
int main(void) {  
  
    float PRO;  
  
    printf("ingrese el promedio del alumno: ");  
  
    scanf("%f", &PRO);  
  
    if (PRO >= 6)  
  
        printf("\nAprobado");  
  
    return 0;}
```

Estructura Condicional - IF - ELSE

`/* El conjunto de instrucciones muestra la
sintaxis de la estructura if-else en C. */`

`if (<condición>)`

`<operación1>;`

`else`

`<operación2>;`

Estructura Condicional - IF - ELSE - ELSE IF

```
/* El conjunto de instrucciones muestra la sintaxis de la estructura  
if-else en C. */
```

```
if (<condición>)
```

```
    <operación1>;
```

```
else if(<condición>)
```

```
    <operación2>;
```

```
else
```

```
    <operación2>;
```

Estructura Condicional - IF - ELSE

```
void main(void){ float PRO;

printf("Ingrese el promedio del alumno: ");

scanf("%f", &PRO);

if (PRO >= 6.0)

    printf("\nAprobado");

else

    printf("\nReprobado");

}
```

Ejercicio I

Desarrollar un programa que me permita ingresar tres números enteros por teclado. Luego determinar cuál de los tres numeros ingresados es el mayor y mostrarlo por pantalla.

Ejercicio II

Desarrollar un programa que me permita ingresar dos notas y determinar cuál de ellas es la mayor y si esta aprobada (se aprueba con 6).

Estructuras Repetitivas

En la práctica, durante la solución de problemas, es muy común encontrar, operaciones que se deben ejecutar un número determinado de veces. Si bien las instrucciones son las mismas, los datos varían. El conjunto de instrucciones que se ejecuta repetidamente recibe el nombre de **ciclo**.

Estructuras Repetitivas

Todo ciclo debe terminar luego de repetirse un número finito de veces. Dentro del conjunto de instrucciones siempre debe existir una condición de parada o fin de ciclo. En cada iteración del mismo son evaluadas las condiciones necesarias para decidir si se debe seguir ejecutando o si debe detenerse.

Estructuras Repetitivas - for

```
for (<contador de ciclos>; <condición> ; <incremento>)  
{  
    proceso; /* cuerpo de la estructura for */  
}
```

Estructuras Repetitivas - for

```
for (int i=0; i<10 ; i=i+1)
{
    printf ("%d",i) ;
}
```

Ejemplo

```
void main(void) {
    int i;
    float sueldo,sueldo_anual;
    sueldo_anual = 0;
    for (i=0; i<=13; i=i+1){
        printf("Ingrese el salario del profesor:");
        scanf("%f", &sueldo);
        sueldo_anual= sueldo_anual + sueldo;
    }
    printf("\nEl sueldo total anual es: %f",
    sueldo_anual);
}
```

Estructuras Repetitivas - while

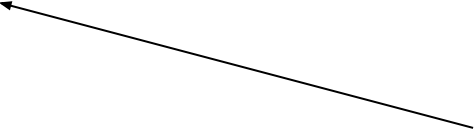
```
while ( <condición> )  
  
    {  
  
    proceso; /* cuerpo de la estructura while */  
  
    }
```

Estructuras Repetitivas - do-while

```
do{  
  
    proceso; /* cuerpo de la estructura do-while */  
  
}while ( <condición> );
```

Estructuras Repetitivas - do-while

```
do{  
  
proceso; /* cuerpo de la estructura for */  
  
}while ( <condición> );
```



Es la única estructura
repetitiva que al menos
se ejecuta una vez.

Estructuras Repetitivas - while

Desarrollar una programa que me permita ingresar 30 notas por teclado. Al final se deberá calcular el promedio general de las notas y mostrarlo por pantalla.

- Determinar cuantas notas están reprobadas(se reprueba con menos de cuatro)
-

Estructuras Repetitivas - while

Desarrollar una programa que me permita calcular el promedio de los números pares y de los números impares comprendidos entre 0 y 100.

- También de se deberá mostrar ambos promedios
-

Ejercicio III

Realizar un programa que me permita ingresar por teclado las notas de 10 alumnos. Se deberá calcular el promedio de todas las notas ingresadas y mostrarlo.

Contadores

Son un tipo de variables que tienen un incremento constante (IC).

$\text{cont} = \text{cont} + \text{IC}$
 $\text{cont} = \text{cont} + 1$

Acumuladores

Son un tipo de variables que tienen un incremento variable (IV).

$$\text{acu} = \text{acu} + \text{IV}$$
$$\text{acu} = \text{acu} + a$$

Ejercicio III

Dado un entero, n , realiza las siguientes acciones condicionales:

- $1 < n \leq 100$
 - Si n es impar, imprime **Extraño**
 - Si n es par y en el rango inclusivo de 2 a 5, imprima **No es Extraño**
 - Si n es par y en el rango inclusivo de 6 a 20, imprima **Extraño**
 - Si n es par y es mayor que 20, imprime **No es Extraño**
-

Ejercicio IV

El ingreso de un entero, n .

- Restricción $1 \leq n \leq 20$
 - Se debe mostrar por pantalla i al cuadrado n veces.
 - Ejemplo:
 - Se Ingresa por teclado un entero: 5
 - Salida por Pantalla: 0 1 4 9 16
-